RaceDay API

A RESTful API built with ASP.NET Core for managing race event registrations, results, and categories.

Setup and Run Instructions

Prerequisites

- .NET 6.0 or later
- SQL Server (LocalDB or full instance)
- Visual Studio 2022 or later

Steps to Run

1. Clone the repository:

git clone https://github.com/mukonaverona27-cell/RaceDay.Api.git

cd RaceDay.Api

2. Update the database connection string:

Open appsettings.json and update the connection string to match your SQL Server instance:

"ConnectionStrings": {

"DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=RaceDayDB;Trusted_Connection=True;MultipleActiveResultSets=true"

}

3. Apply Database Migrations:

Open the Package Manager Console in Visual Studio and run:

Update-Database

4. Run the API:

Press F5 in Visual Studio or use the command:

dotnet run

The API will be available at https://localhost:70xx and http://localhost:50xx

5. Access Swagger UI:

Navigate to https://localhost:xxxx/swagger in your browser to test the API endpoints.

User Roles

Organiser

• Create, update, and delete race events and categories

• View all participant information and race results

• Full system access to all endpoints

Participant

• View available race information and categories

• Register for upcoming races

• View personal race results

• Limited access to authorised endpoints only

Project Structure

```
RaceDay.Api/
├─── Controllers/          # API endpoints
├─── DTOs/                 # Data transfer objects
├─── Data/                 # Database context and migrations
├─── Models/               # Database entity models
├─── Migrations/           # Database migration files
├─── Program.cs            # Application entry point
└─── appsettings.json      # Configuration file
```

Authentication

This API uses JWT Bearer Tokens for secure authentication.

• Register endpoint: /api/auth/register — requires email, password, and role selection

• Login endpoint: /api/auth/login — requires email and password, returns a JWT token

• Include the token in the request header for protected endpoints:

Authorization: Bearer {your-token-here}

Passwords are automatically hashed using ASP.NET Core Identity. User ID and role are stored within the token for server-side session management. All protected endpoints enforce role-based authorisation correctly.

Unit Tests

• Test Project: RaceDay.Api.Tests

• Run tests via Visual Studio Test Explorer → Run All Tests

• Tests cover authentication logic and core endpoint functionality

• All tests pass locally and through the CI/CD pipeline

CI/CD — GitHub Actions

Workflow file location: .github/workflows/ci.yml

The CI pipeline automatically restores dependencies, builds the solution, and runs all unit tests on every commit pushed to the repository. This ensures code quality and functionality are validated continuously.